TB024/75.29 - TDA

Teoría de Algoritmos

Trabajo Práctico n.º 1

Nombre del grupo: Métase quien quiera
Github: FranCrisFIUBA/TDA-TP1

Integrantes:

Padrón	Apellidos	Nombres
113877	Cristaldo	Franco
97644	Ribeiro	Federico Edgardo
98675	Lahuta	Brian Ezequiel
99513	Cori	William

Fecha de entrega	13/04/2026
Entrega número	1

Parte 1: Seleccionando turnos.

1.

Cada turno tiene dos variables, horario límite y beneficio. El objetivo es maximizar el beneficio total, por lo cual el algoritmo deberá priorizar las tareas con mayor beneficio sobre las de menor beneficio.

Método no óptimo 1:

Un algoritmo no óptimo sería aquel que no tenga una correcta elección heurística greedy de los turnos prioritarios, en este caso podría ser aquel que priorice los turnos con horario límite menor, antes que los de mayor beneficio.

Este algoritmo podría brindar soluciones óptimas en ciertos casos, por ejemplo cuando todos los turnos puedan ser completados al no haber solapamientos, en este caso, el beneficio máximo posible sería la suma de todos los beneficios, el cual sería también el beneficio máximo obtenido con el algoritmo.

Como contraejemplo, en caso de haber múltiples turnos

Método no óptimo 2:

Otro algoritmo no optimo sería ordenar los turnos por beneficio, de mayor a menor, y luego intentar cumplirlos en ese orden descartando los que no se lleguen a cumplir por límite horario
Contraejemplo:
Dada la siguiente entrada [(1, 10), (3, 50), (2, 30)] este algoritmo responderia ([1, 2), 80) siendo mejor opcion ([0, 3, 2], 90)

2.

Revisar: La propuesta es, ordenar los turnos por beneficio, de mayor a menor, y asignarlos al horario más tardío posible, de esta forma se guardan los bloques más tempranos para aquellos turnos que deban realizarse antes y se logra seleccionar la mayor cantidad de turnos con beneficio mayor.

Pseudocódigo

```
// Indexado basado en 0

// Parámetro
Sea turnos una lista de tuplas (horario_limite, beneficio)

Sea indice_turnos la agrupación de los turnos con su índice (índice, turno)

Se ordena indice_turnos según el beneficio del turno

Sea H las horas por jornada = 24
// representa los turnos a realizar en cada hora, -1 significa que esa hora está libre
Sea indice_tarea_por_hora un array de tamaño H poblado con -1

Sea beneficio_total = 0

Por cada indice, (horario_limite, beneficio) en indice_turnos
   // Ocupa el bloque más tardío
   // Si un bloque está ocupado, necesariamente es por un turno con mayor beneficio
   Sea hora = horario_limite - 1
   Mientras hora >= 0
      Si indice_tarea_por_hora[hora] != -1
         beneficio_total += beneficio
         indice_tarea_por_hora[hora] = indice

Sea orden una lista vacía
Por cada indice en indice_tarea_por_hora
   Si indice != -1
      orden.append(indice)

Devolver (orden, beneficio_total)
```

3.

- **Óptimo global**: Es la mejor solución entre todas las soluciones posibles del problema, se construye en base a las soluciones óptimas locales. En este caso es el orden de turnos compatibles a realizar que generen el beneficio máximo.
- **Óptimo local**: Es la mejor solución de un subproblema del problema general. En este caso el óptimo local es asignar en el horario más tardío disponible, el turno sin asignar con mayor beneficio.
- **Subestructura óptima**: Es una propiedad de un problema, si el problema tiene subestructura óptima significa que puede dividirse en subproblemas y la solución óptima del problema puede construirse en base a las soluciones óptimas de estos subproblemas. En este caso es
- **Elección greedy**: Es un criterio que permite seleccionar la mejor solución óptima local de cada subproblema del problema. En este caso es elegir los turnos en base a cual tenga mayor beneficio y se da al ordenar los turnos por beneficio de mayor a menor antes de iterar.

4.

Volviendo al pseudocodigo:

Pseudocódigo

// **Las complejidades se analizan en el peor caso**
 	
Sea turnos una lista de tuplas (horario_limite, beneficio)
 	
// temporal: **O(n)**
// espacial: **O(n)**
Sea indice_turnos la agrupación de los turnos con su índice (índice, turno)
 	
// temporal: **O(n log n)**
// espacial: dependiendo la implementación, en python es **O(n)**
Se ordena indice_turnos según el beneficio del turno
 	
Sea H las horas por jornada = 24
// temporal: **O(H)**
// espacial: **O(H)**
Sea indice_tarea_por_hora un array de tamaño H poblado con -1
 	
Sea beneficio_total = 0
 	
// temporal: **O(n * H)**
Por cada indice, (horario_limite, beneficio) en indice_turnos
  // Ocupa el bloque más tardío
  // Si un bloque está ocupado, necesariamente es por un turno con mayor beneficio
  Sea hora = horario_limite - 1
  Mientras hora >= 0
  Si indice_tarea_por_hora[hora] != -1
  beneficio_total += beneficio
  indice_tarea_por_hora[hora] = indice
 	
Sea orden una lista vacía
// temporal: **O(H)**
// espacial: **O(H)**
Por cada indice en indice_tarea_por_hora
  Si indice != -1
  orden.append(indice)
 	
Devolver (orden, beneficio_total)

Complejidad temporal.

Complejidad temporal:
O(n + n log n + H + n*H + H)
agrupando y simplificando constantes (H):
O(n + n log n)
nos quedamos con la mayor complejidad:
O(n log n)

Complejidad espacial.

Complejidad espacial:
O(n + n + H + H)
agrupando y simplificando constantes (H) nos queda:
O(n)

5.

Si iteramos por cada uno de los turnos ordenados por beneficio de forma decreciente y buscamos bloque más tardío disponible compatible con el turno

- **Si el bloque existe**: le asignamos el turno
- **Si el bloque no existe**: significa que ya se asignó un turno **necesariamente más beneficioso** en todos los bloques que pudo haber ocupado el turno actual, por lo tanto el turno no vale la pena y no es asignado a ningún bloque. Esto garantiza que sean asignados la mayor cantidad de turnos lo más beneficiosos posibles. Esta es la solución óptima local.

6.

Solución en Python[1]

```
def main(turnos: list[tuple[int, int]]) -> tuple[list[int], int]:
    hora_maxima = 24
    indice_tarea_por_hora: list[int] = [-1] * hora_maxima

    indice_turnos = [(i, turno) for i, turno in enumerate(turnos)]
    indice_turnos.sort(key=lambda x: x[1][1], reverse=True)

    ganancia_total = 0
    for indice, (hora_limite, ganancia) in indice_turnos:
        hora = hora_limite - 1
        while hora >= 0:
            if indice_tarea_por_hora[hora] == -1:
                indice_tarea_por_hora[hora] = indice
                ganancia_total += ganancia
                break

            hora -= 1

    orden: list[int] = []
    for i in indice_tarea_por_hora:
        if i != -1:
            orden.append(i)

    return orden, ganancia_total
```

[1] Solucion Ejercico 1

Parte 2: Multiplicación de polinomios.

1.

El algoritmo tradicional para multiplicar dos polinomios funciona de manera directa multiplicando cada término de $A(x)$ por cada término de $B(x)$, y después sumando los productos que van a parar a la misma potencia de x.

Si $A(x) = \sum_{i=0}^{n-1} a_i x^i$ y $B(x) = \sum_{j=0}^{n-1} b_j x^j$, entonces el resultado es otro polinomio $C(x) = \sum_{k=0}^{2n-2} c_k x^k$, donde cada coeficiente c_k se arma sumando todos los productos $a_i b_j$ y se lo acumula en la posición $i+j$ del resultado. Como hay que considerar todos los pares posibles de coeficientes, en total se hacen n^2 multiplicaciones, y además una cantidad del mismo orden de sumas. Por eso, la complejidad temporal del algoritmo es $\Theta(n^2)$.

2.

Para disminuir la cantidad de multiplicaciones, se puede usar un enfoque de división y conquista basado en Karatsuba. Como n es potencia de 2, se parte cada polinomio en dos mitades de tamaño $n/2$. Entonces quedan: $A(x) = A_1 x^{\frac{n}{2}} + A_0$ y $B(x) = B_1 x^{\frac{n}{2}} + B_0$.

Si hacemos el producto de manera directa, queda: $A(x)B(x) = A_1 B_1 x^n + (A_0 B_1 + A_1 B_0) x^{\frac{n}{2}} + A_0 B_0$. El problema es que, si seguimos esa fórmula tal cual en forma recursiva, terminamos teniendo que hacer cuatro multiplicaciones: $A_1 B_1$, $A_0 B_0$, $A_0 B_1$ y $A_1 B_0$.

Sin embargo, encontramos una mejora al ver que: $(A_0 + A_1)(B_0 + B_1) = A_0 B_0 + A_0 B_1 + A_1 B_0 + A_1 B_1$. Luego, en vez de calcular por separado los dos términos cruzados, los obtenemos despejando: $A_0 B_1 + A_1 B_0 = (A_0 + A_1)(B_0 + B_1) - A_0 B_0 - A_1 B_1$.

Así, en lugar de hacer cuatro multiplicaciones recursivas, alcanza con hacer solo tres: $A_0 B_0$, $(A_0 + A_1)(B_0 + B_1)$ y $A_1 B_1$. Después, armamos el resultado final como: $C(x) = A_1 B_1 x^n + ((A_0 + A_1)(B_0 + B_1) - A_1 B_1 - A_0 B_0) x^{\frac{n}{2}} + A_0 B_0$.

3.

El algoritmo que propusimos parte cada polinomio en dos mitades y, en vez de hacer las cuatro multiplicaciones recursivas que saldrían de forma directa, resuelve el problema con solo tres: A_0B_0, $(A_0 + A_1)(B_0 + B_1)$ y A_1B_1. Después, con algunas sumas, restas y multiplicaciones por potencias de x, arma el resultado final.

Como todas esas operaciones extra se hacen sobre arreglos de coeficientes de tamaño proporcional a n, su costo es $\Theta(n)$. Por eso, la recurrencia del algoritmo queda:

$$T(n) = 3T(n/2) + \Theta(n), T(1) = \Theta(1).$$

Para resolverla usamos el teorema maestro, con $a = 3$, $b = 2$ y $f(n) = \Theta(n)$. Como $n^{log\ 2^3}$ crece más rápido que n, corresponde aplicar el caso 1. Entonces, la complejidad temporal del algoritmo es:

$$T(n) = \Theta(n^{log\ 2^3}),$$

es decir, aproximadamente $\Theta(n^{1.59})$.

Parte 3: Sobrevivir al reto.

1.

cada caja se calcula cuál es la mayor altura de torre que puede construirse comenzando con esa caja, probando poner encima todas las cajas que sean más grandes en largo y ancho; cada vez que se intenta poner una caja arriba, se suma su altura a la mejor torre posible que comienza en esa caja superior, y ese resultado se memoiza para no volver a calcularlo si se necesita otra vez. Al final se evalúa comenzar la torre desde cada caja y se elige la altura máxima obtenida. De esta forma la mejor torre posible se reconstruye utilizando la mejor torre que comienza en cada caja.

2.

Sea OPT(i) la altura máxima de una pila cuya caja superior es i
OPT(1) = y_1
OPT(i) = y_i + max(OPT(j) tal que x_j < x_i y z_j < z_i)

3.

Pseudocodigo

Ordenar cajas por (x, z)
 	
Por i = 1..n
  OPT[i] = y[i]
 	
Por i = 1..n
  Por j = 1..i-1
  Si x[j] < x[i] y z[j] < z[i]
  OPT[i] = max(OPT[i], OPT[j] + y_i)
 	
Devuelve max(OPT)

4.

Volviendo al pseudocódigo:

Pseudocódigo

// temporal: O(n log n)
// espacial: O(n)
Ordenar cajas por (x, z)
 	
// temporal: O(n)
// espacial: O(n)
Por i = 1..n
  OPT[i] = y[i]
 	
// temporal: O(n^2)
Por i = 1..n
  Por j = 1..i-1
  Si x[j] < x[i] y z[j] < z[i]
  OPT[i] = max(OPT[i], OPT[j] + y_i)
 	
Devuelve max(OPT)

Complejidad espacial.

O(n + n)
agrupando: **O(n)**

Complejidad temporal.

O(n log n + n + n^2)
nos quedamos con la mayor complejidad: **O(n^2)**

5.

Solución en Python[2]

```
def main(
    cajas: Sequence[tuple[float, float, float]]
) -> tuple[float, list[tuple[int, tuple[float, float]]]]:

    indices_cajas = [(indice, caja) for indice, caja in enumerate(cajas)]
    indices_cajas.sort(key=lambda c: (c[1][0], c[1][2]))  # (x, z)
    n = len(indices_cajas)
    opt = [0.0] * n
    indice_caja_superior_anterior = [-1] * n

    for i in range(n):
        _, (x_i, y_i, z_i) = indices_cajas[i]
        opt[i] = y_i

    for i in range(n):
        indice_i, (x_i, y_i, z_i) = indices_cajas[i]

        for j in range(i):
            indice_j, (x_j, y_j, z_j) = indices_cajas[j]

            if x_j < x_i and z_j < z_i:
                if opt[j] + y_i > opt[i]:
                    opt[i] = opt[j] + y_i
                    indice_caja_superior_anterior[i] = j

    mejor_caso = max(range(n), key=lambda i: opt[i])
    altura_maxima = opt[mejor_caso]

    pila = []
    i = mejor_caso
    while i != -1:
        indice, (x, y, z) = indices_cajas[i]
        pila.append((indice, (x, z)))
        i = indice_caja_superior_anterior[i]

    return altura_maxima, pila
```

[2] Solucion Ejercicio 3

6.

Sí, existen casos en los que hay más de una única solución óptima para las mismas cajas, como ejemplo:

Entrada: [(1, 2, 1), (2, 3, 2), (3, 3, 3), (4, 5, 4)]
Altura máxima: 10
Torre óptima 1: [3, 1, 0]
Torre óptima 2: [3, 2, 0]

En este caso no se puede apilar las cuatro cajas al mismo tiempo y existen dos cajas (1 y 2) de igual altura que pueden apilarse bajo 0 y sobre 3, lo que nos da dos instancias distintas de torres que llegan a una solución óptima.